

DD2459 Software Reliability

Lecture 6

Advanced Topics in Testing

Part 1: Mutation Testing

(see Amman and Offut,
Chapters 9.1.2, 9.2.2)

Mutation Testing

- Mutation testing concerns:

evaluation of test suites for their **test effectiveness**,
i.e. ability to reveal errors.

- Need an **objective** method to measure effectiveness
- Relationship between coverage and effectiveness is unclear
- Need to define “**what is an error**” (i.e. **fault model**)
- Basic idea is to **inject defined errors** into the SUT and evaluate whether a given test suite finds them.
- “***Killing a mutation***”

Basic Idea

- We can **statistically estimate** (say) fish in a lake by releasing a number of marked fish, and then counting the number of marked fish in a subsequent small catch.
- Example: release **20 fish**
- Catch **40 fish, 4 marked**.
- Then **1 in 10** is marked, so estimate **200 fish**
- Pursue the same idea with “marked SW fish”?

Mutations and Mutants

- The “marked fish” are **injected errors**, termed **mutations**
- The (entire) **mutated code** is termed a **mutant**
- E.g: replace **<** by **>** in one Boolean expression
`if (x < 0) then`
becomes
`if (x > 0) then`
- If test suite finds mutation we say this particular mutant is **killed**
- Make large set of mutants – using a checklist of known mutations with a **mutation tool**, e.g. **muJava**

Higher-order Mutations

- A single mutation of one code component (e.g. assignment) is called a **first-order mutant**.
- A **second-order mutant** is obtained by applying a mutation to a first-order mutant
- Gives **n-th order mutants** for all $n > 0$

Mutation score

- Idea is that if we can kill a mutant we can identify a real bug too
- Mutants which are behaviourally equivalent to the original code are called **equivalents**
- Clearly cannot kill equivalents
- Mutation score % =

Number of killed mutants

total number of non-equivalent mutants * 100

Why mutation works

- Two assumptions (hypotheses) are used in this field

(1) the competent programmer hypothesis

i.e. “the program is mostly correct”

and

(2) the coupling effect

Assumption 1: Competent Programmer Hypothesis (CPH)

- CPH claims that since most professional programmers are *competent*, most of their code is correct.
- CPH claims that only a small number of errors exist in the form of reversible mutations
- i.e. *the program is a n -th-order mutant of some correct code.*
- n is presumably small?

Assumption 2:

Coupling Effect (CE)

(Hypothesis)

- The **competent programmer hypothesis** limits the testing problem to detecting specific errors.
i.e. “**detect n -th-order mutants**”
- This problem is still *too large, n is unbounded*
- The **coupling effect** claims that higher-order mutants are *coupled* to lower-order mutants.
- In particular, **detecting first-order mutants detects n -th order for any n .**
- Some limited empirical evidence for this (Offut 1992)

Problem:

Equivalent Mutants

- A mutant is **equivalent** if it does not change the behaviour of the code
- E.g. $(x + y) \rightarrow (y + x)$
- Equivalent mutants skew the mutation score lower
- Difficult to know by how much!
- Offut and Pan [1997] estimated 9%
- Bybro [2003] KTH Masters thesis 8%
- Static analysers can detect about 50% of these
- Use theorem proving (verification) techniques

Early Approach:

22 Fortran mutations

AAR Array reference for array reference replacement

ABS Absolute value insertion

ACR Array reference for constant replacement

AOR Arithmetic operator replacement

ASR Array reference for scalar replacement

CAR Constant for array reference replacement

CNR Comparable array name replacement

CRP Constants replacement

CSR Constant for Scalar variable replacement

DER Do statement End replacement

DSA Data statement alterations

GLR Goto label replacement

LCR Logical connector replacement

ROR Relational operator replacement

RSR Return statement replacement

SAN Statement analysis

SAR Scalar for array replacement

SCR Scalar for constant replacement

SDL Statement deletion

SRC Source constant replacement

SVR Scalar variable replacement

UOI Unary operator insertion

Recent Approach:

Java Mutation Operators

- First letter is *mutation category*
- A = access control
- E = common programming mistakes
- I = inheritance
- J = Java-specific features
- O = method overloading
- P = polymorphism

AMC Access modifier change
EAM Accessor method change
EMM Modifier method change
EOA Reference assignment and
content assignment replacement
EOC Reference comparison and
content comparison replacement
IHD Hiding variable deletion
IHI Hiding variable insertion
IOD Overriding method deletion
IOP Overriding method calling position change
IOR Overridden method rename
IPC Explicit call of parent's constructor deletion
ISK super keyword deletion
JDC Java supported default constructor create

JID Member variable initialisation deletion

JSC static modifier change

JTD this keyword deletion

OAQ Argument order change

OAN Argument number change

OMD Overloaded method deletion

OMR Overloaded method contents change

PMD Instance variable deletion with parent class type

PNC new method call with child class type

PPD Parameter variable declaration with child class type

PRV Reference assignment with other compatible type

Practical Example

- Myer's Triangle program (Lab 1)
- Myers "complete" test suite
- (c.f. Lab 1: final 13 test requirements)
- Bybro [2003] Java mutation tool and code
- 88% mutation score, 96% statement coverage

Summary

- Mutation testing introduces a simple **fault model** into software testing theory
- Model has some correspondence with **real-life errors**
- Based on two hypotheses: **CPH** and **CE**
- **Mutation score** grades a test suite for effectiveness
- **Mutation tools** exist
- Not yet caught on in **industry** but interesting

Status of Mutation Testing

- Various strategies: weak mutation, interface mutation, specification-based mutation
- Our version is called strong mutation
- Many mutation tools available e.g. muJava
- Cost of generating mutants and detecting equivalents has come down
- Not yet widely used in industry
- Still considered “academic”, not understood?

DD2459 Software Reliability

Lecture 6

Advanced Topics in Testing

Part 2: Machine Learning for Testing

(see e.g. Bennaceur, Hähnle, Meinke, *Machine Learning for Dynamic Software Analysis*, Springer 2016)

ML for Testing

- Machine learning (ML) can automate many software engineering tasks: e.g. testing.
- **Advantages:** *flexible, intelligent, data driven*
- **Disadvantages:** *difficult to train, lack of training data, ML is data hungry.*
- But testing *generates* a lot of data, so might be good in the future?
- Route to **full test automation** (and generating even more data) ??

Case Study 1:

Black-box Checking

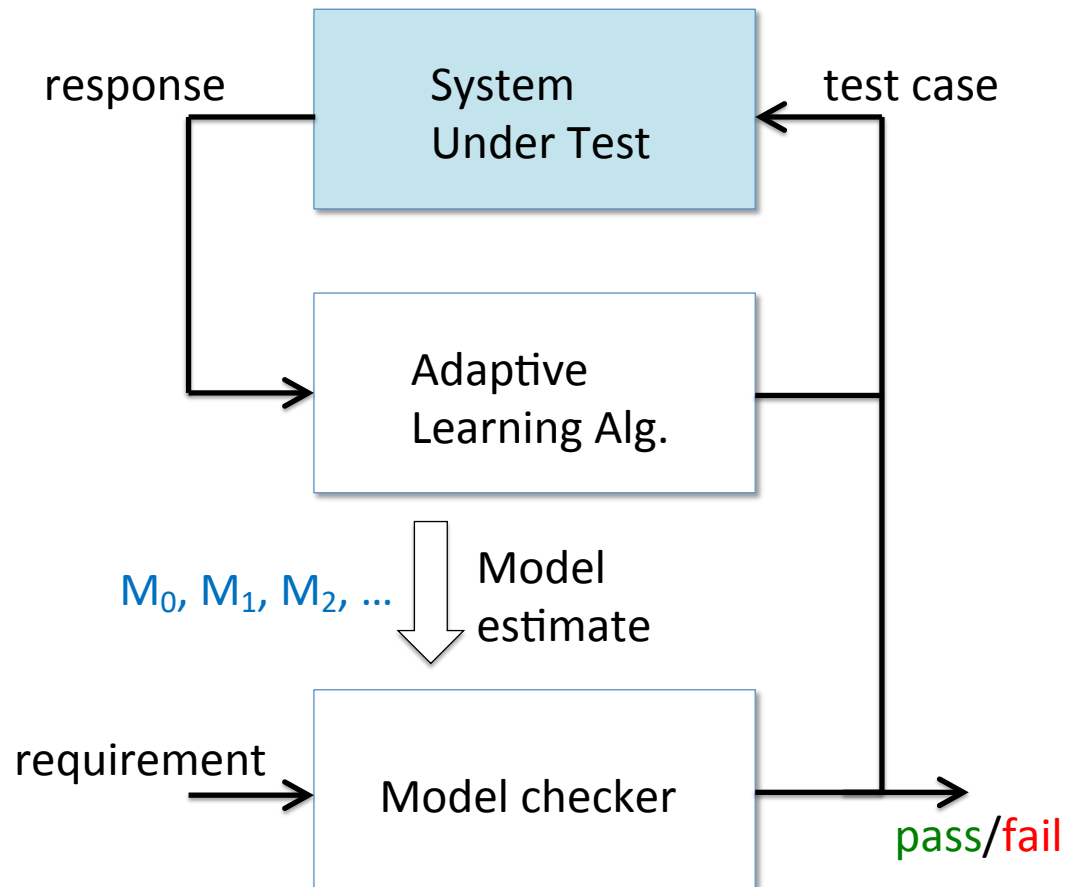
- Black-box checking (aka. *learning-based testing LBT*) is a **black-box** testing method
- Focuses on **requirements testing**
- Can view testing as a **search problem**
- Black-box checking uses **active ML** as a **search heuristic**
- Guided by analogies between **learning and testing**

Key Testing Problem: Test Optimisation

Can we use ML to..

1. Dynamically generate test cases $t_1, t_2, \dots$
2. Iteratively **improve** the next test case t_{n+1} on the basis of $t_1, \dots, t_n$
3. Quickly execute a **large number** of tests?
4. Obtain **good coverage**?
5. Find bugs **quickly**?

LBT architecture



Basic Idea ...

LBT is a **search heuristic** that:

1. **Actively** and **incrementally** learns an **SUT model** (automaton model)
2. Uses **inductive learning principles** aka. **generalisation** to **predict** bugs
3. Uses a **best prediction** as **next test case** t_{n+1}
4. **Refines** models $M_0, M_1, M_2, \dots$ according to test outcomes.
5. Model sequence $M_0, M_1, M_2, \dots$ **converges** to SUT

Case Study:

Testing autonomous driving

- The film clip below shows a case study of using a learning-based technique to test the **software of an autonomous vehicle**.
- Uses a **virtual physical environment** (weather!)
- ML systematically tests thousands of instances of the same **use case : *4-vehicle overtaking***.
- Fast, good coverage, effective and safe!

en Download Save Settings Road

New Open from Pool ODIC File

Open Download Save Activate Properties Scenario

Manage Traffic Objects

Save Save As File

Complete Scene Synchronization

Road Fellows Customize

1 Configuration page 2 Slalom 3 StraightLong 4 PedestriansCrossing 5 acc_test

Python Scripts
GenerateSensorMotion
Experiment_001
Simulation Model
ASM_Traffic.sbx
ASM_Traffic.sdi
Python Scripts
Road
StraightLong.rd
Scenario
acc_test.xml
Simulation Results

Location: C:\Users\hojat\Downloads\ESIM-MDL
3/6/2009 1:24:36 PM

ription:

Properties

Search properties

Traffic scenario

Name	acc_test
Description	
Author	
Creation date	01/31/2020 16:19:51

References

Road	StraightLong.rd
Change road	...
Reload road	Reload
Remove road	Remove

Consistency

Valid

1 Maneuver F 7 Fellows U 0 User Signals

Search

F3 Fellow3

Duration (0 s)

Distance [m]
M1 | Constant: 80 m

Lane selection
Constant: 0

1 Immediate

Velocity
Constant: 50 km/h

Continue

2 Immediate

F4 Dummy Walk

Duration (1 s)

Position
Constant: 200 m

Lateral deviation
Constant: 7 m

1 Distance (F1 to M1, 80 m), ...

Continue

Lateral deviation
Ramp: -3 m

2 Duration (20 s)

F5 Dummy Walk

Duration (1 s)

Position
Constant: 400 m

Lateral deviation
Constant: -3 m

1 Distance (F2 to M1, 80 m), ...

Continue

Lateral deviation
Ramp: 7 m

2 Duration (20 s)

F6 Fellow3

Duration (1 s)

Position
Constant: 50 m

Lane selection
Constant: 0

1 Immediate

Velocity
Constant: 20 km/h

Lateral deviation
Constant: -3 m

2 Duration (20 s)

Pages

Errors 6 Warnings 760 Messages

Search

Severity	Module	Time	Message
Info	VEOS	14:07:46.188	Simulated from 46332.199 s to 46471.277 s with the following real-time acceleration factor: 0.4.
Info	MDLDParm...	14:49:02.617	The 'start' command of the maneuver control succeeded.
Info	MDLDParm...	14:53:08.288	The 'start' command of the maneuver control succeeded.
Info	MDLDParm...	14:53:24.511	The 'stop' command of the maneuver control succeeded.

Case Study 2:

Test Requirement Identification

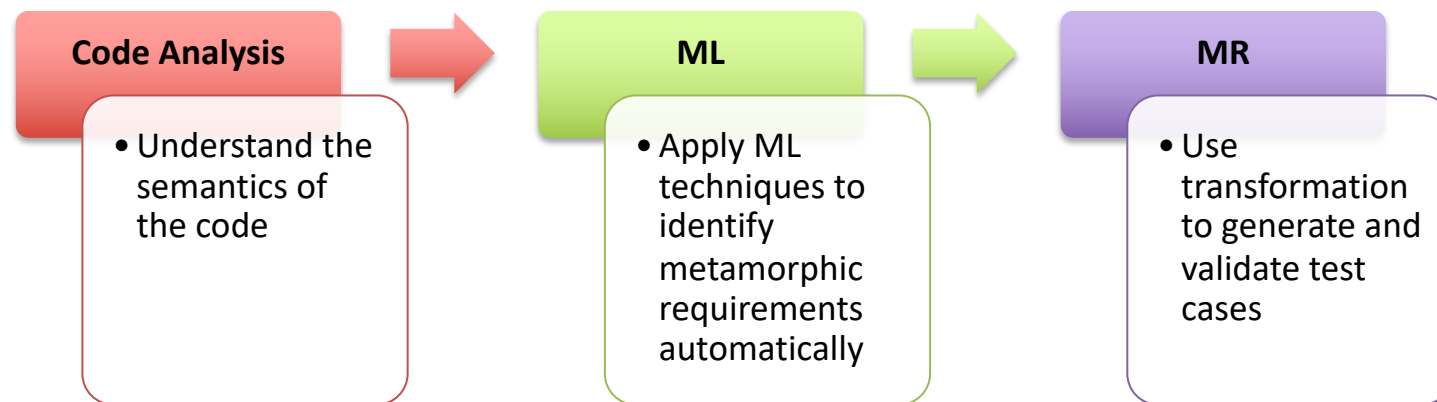
Key Testing Problem. Test requirements are hard to identify and may quickly change

Can we use ML to..

1. Automatically identify the correct requirements to test from the code itself?
2. Automatically generate and execute a **large number** of tests of these requirements?

Learning Metamorphic Requirements

- Use ML to identify the **metamorphic requirements (Lecture 4)** applicable to a program.



SVM Learning: finding an optimal decision boundary

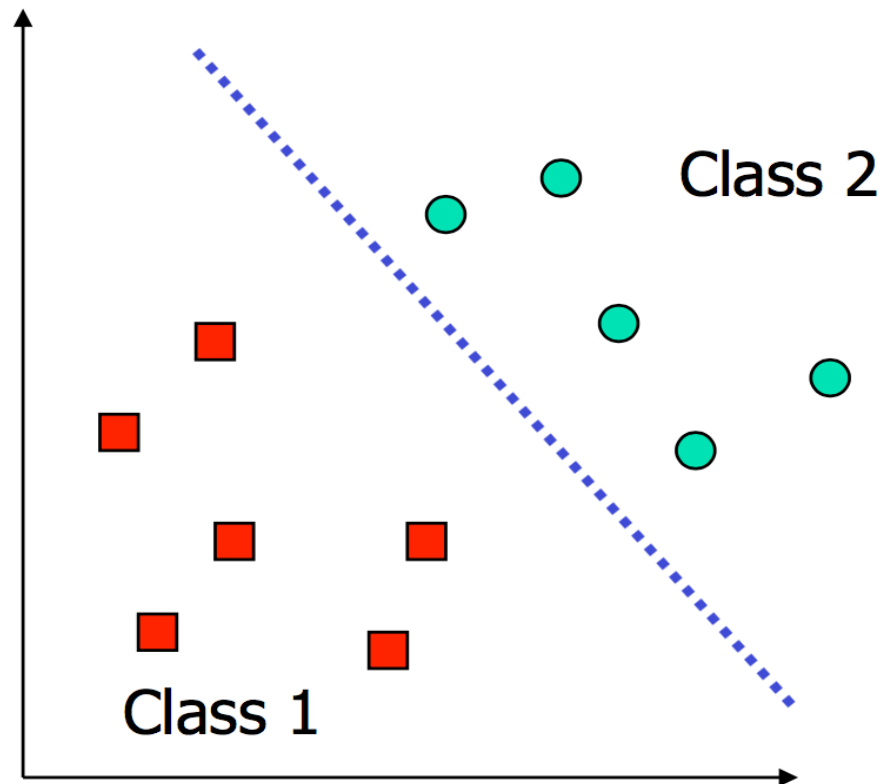
Consider a binary classification problem, over a linearly separable set of data points

Clearly many decision boundaries

What is the “best”?

Green points are programs that satisfy metamorphic property P .

Red points are programs which don't.



Experimental Evaluation

(Nair, Meinke, Eldh 2019)

- **Metamorphic Requirement “Inclusive Law”**: The program output increases or remains constant after adding a new element to input.
- More precisely, for a *list* input $[x_1, \dots, x_n]$

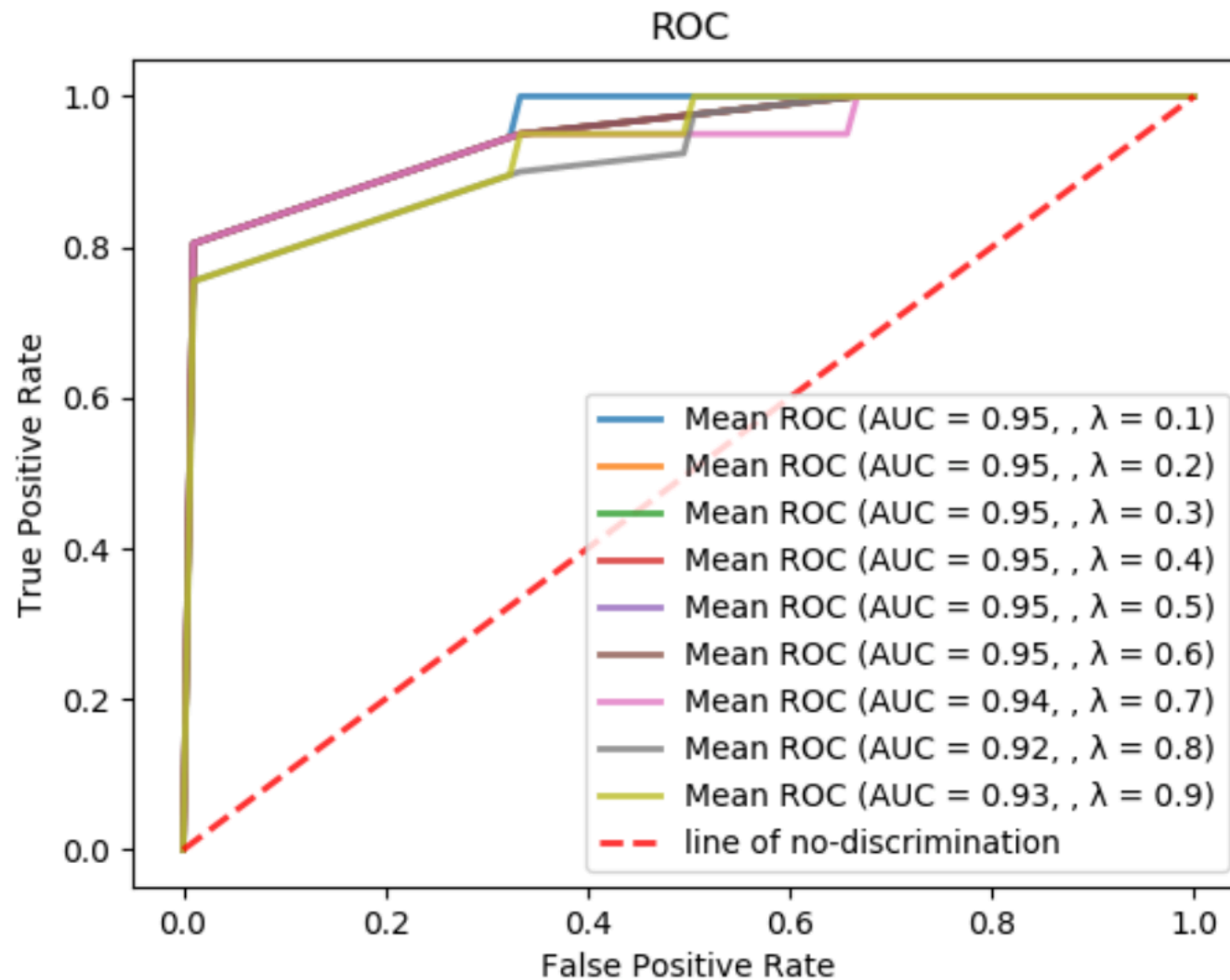
$$f([x_1, \dots, x_{n+1}], y) \geq f(T([x_1, \dots, x_{n+1}]), y)$$

where

$$T([x_1, \dots, x_{n+1}]) = [x_1, \dots, x_n]$$

Learning Results

ROC curve



Lecture 6 Part 2:

Summary

- ML is an important next step forwards in **software engineering automation**, including **test automation**
- Currently an *active research area*
- Need to understand the “*paradigm*” of ML
- Many **types of ML algorithms** are relevant
- Many **routine** test activities might be automated
- ML will always be **data hungry**
- **Lack of data** might be the main **bottleneck**

Lecture 6: Summary

- Testing technology is undergoing a lot of change
- Probably won't do things in the future the way we do now
- Mutation testing is a way to measure the quality of a test suite.
- It introduces a fault model into testing.
- Machine learning can potentially automate different testing tasks in a flexible, intelligent and data driven way.
- Testing throws up a lot of data, so could be good (ML is data hungry)

DD2459 Course Summary

- Testing is a **big field**
- Testing is an **old field**: techniques from 1950s
- Testing is a **new field**: MBT, model checkers, ML, etc.
- In DD2459 we have *focused* on:
 - Glass box
 - Black-box
 - Requirements
 - Model-based
 - Automated TCG

Major Course Omissions

- OO testing
- GUI testing
- Integration/system testing
- Agile testing / test-driven development

Now you can **teach yourself** many of these based on our classes and the course textbook, or other sources!

Testing technology is still a work in progress!